IDs: 213851579, 213310485

Video Processing – HW2

Optical Flow and Video Stabilization

Course 0512.4263 2025/2026

Section 1 – General Questions

Recall the optical flow formula:

$$0 = I_t + I_x u + I_y v$$

For each pixel: two unknowns (u, v), one equation.

Q1

What is the constant brightness constraint? In your answer relate to:

a. How does it help us to solve the optical flow between two images?

b. Is this assumption correct in real world scenarios? (what happens when there are reflective objects in the image?).

Answer.

The *constant brightness constraint* assumes that the intensity of a physical point in the scene does not change between two consecutive frames; the point only *moves*. Formally, if a point at (x, y) in frame t moves by (u, v), then

$$I(x, y, t) = I(x + u,\ y + v,\ t + 1).$$

a. **How it helps.** A first-order Taylor expansion of the right-hand side, keeping only linear terms, gives

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + I_x u + I_y v + I_t,$$

so the constraint becomes $I_x u + I_y v + I_t = 0$. This yields *one* linear equation per pixel that ties the unknown flow (u, v) to the measurable spatial and temporal gradients I_x, I_y, I_t.

b. **Correct in the real world?** Not always. It holds only under stable illumination and for Lambertian (matte) surfaces. It is violated by lighting changes, shadows, and especially *reflective / specular* objects.

Q2

What is the aperture problem and what can we do about it?

Answer.

One equation, two unknowns per pixel, so only the flow component along the gradient (normal to an edge) is recoverable; the component along the edge is unknown. Through a small window, motion parallel to an edge is invisible.

Solution: assume neighboring pixels share the same flow, stack their equations over a window, solve by least squares (Lucas–Kanade). Needs gradients in two directions (corners/texture), not a single edge.

Q3

How did Lucas-Kanade solve the optical flow problem? (what did they assume about the movement of each pixel?). Hint: it is related to the movement of the neighbourhood of each pixel.

Answer.

They assumed all pixels in a small window around a point share the same flow (u, v) (locally constant motion). Each of the n pixels then gives one brightness-constancy equation, yielding n equations for two unknowns, an overdetermined system solved by least squares: $(u, v) = (A^\top A)^{-1} A^\top b$. This resolves the aperture problem when the window is textured.

Q4

Is the Lucas-Kanade assumption true around the object boundaries? Why?

Answer.

No. At a boundary the window straddles two objects with different motions (e.g. foreground vs. background), so the single-flow assumption breaks and least squares averages the two into a wrong vector.

Q5

Propose a general idea how to correctly find the optical flow on the object's boundaries, given you can get any input you desire (except from the true movement of each pixel). For example, you can get a depth map / label image (you know for each pixel to which object it belongs to or what is its depth). Write which inputs you assume to have and the general idea of your solution.

Answer.

Assumed input: a per-pixel label map giving each pixel's object id.

Idea: make the LK window respect object boundaries. When solving for a pixel, mask the window to keep only neighbors with the *same* label as the center pixel, and run least squares on those alone. This removes the other-object pixels that broke the single-motion assumption, so each window contains one coherent motion and the flow is correct up to the boundary.

Section 2 – Lucas-Kanade Optical Flow

Q4.1

Put the image created in `river_results/0_river_one_LK_step_result.png` in your PDF and explain the result.

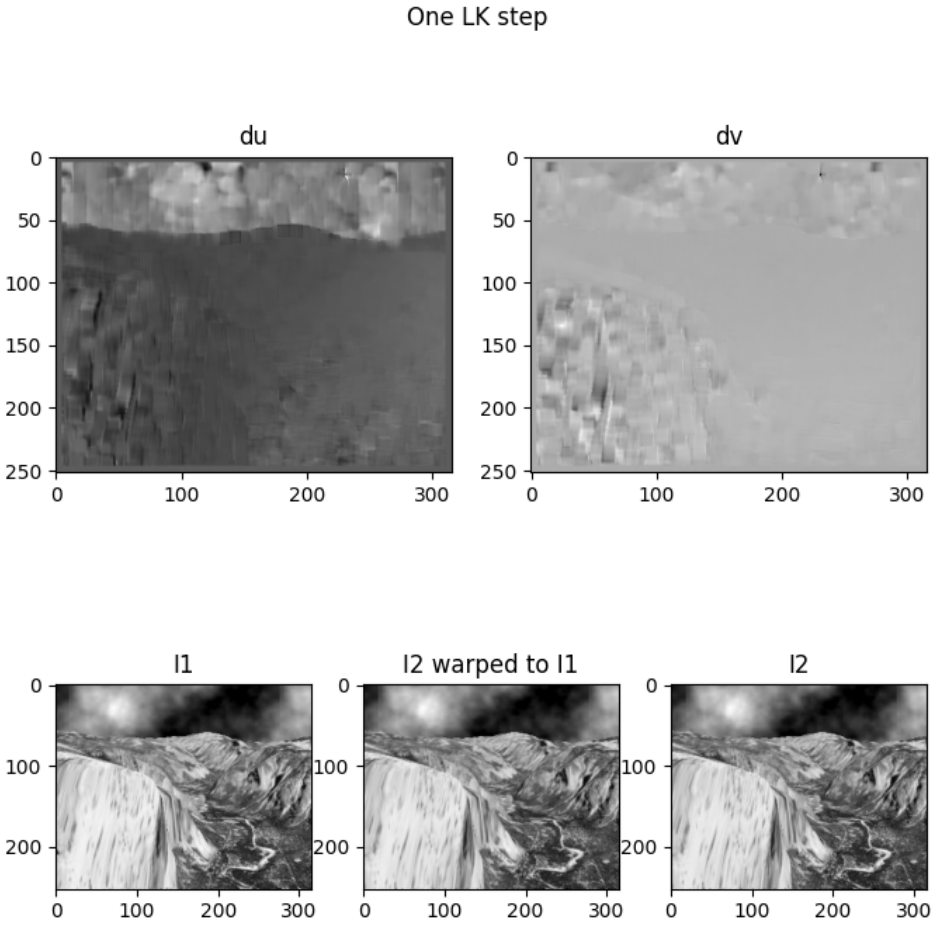

Figure 1: One LK step result.

Explanation.

The top row shows the estimated flow du, dv; the bottom row shows I_1, the back-warped I_2, and I_2. In the static, textured regions (rocks and banks) the warped I_2 matches I_1 well. In the fast sky moving the flow is large and noisy, so the warp there is broken and full of holes. As a result the MSE actually increases after one step: the sky moves too much for a single step to handle.

Q4.2

Look at the gifs created in `river_results/2_after_one_lk_step.gif` and `river_results/1_original.gif` and answer the following question in the report: Why is the result imperfect?

Answer.

The single step uses a first-order Taylor approximation of the brightness constraint, which is only valid for small motion. The river's displacement is too large, so the step cannot reach the true shift in one go. Low-texture areas are also ambiguous (aperture problem), and without a coarse-to-fine pyramid large motion simply cannot be captured. This is why the full pyramidal LK is needed next.

Q6

Run the entire `main_river.py` script. Put all png outputs in your PDF report. Explain all results. Explain why your gif `3_after_full_lk.gif` looks better now.

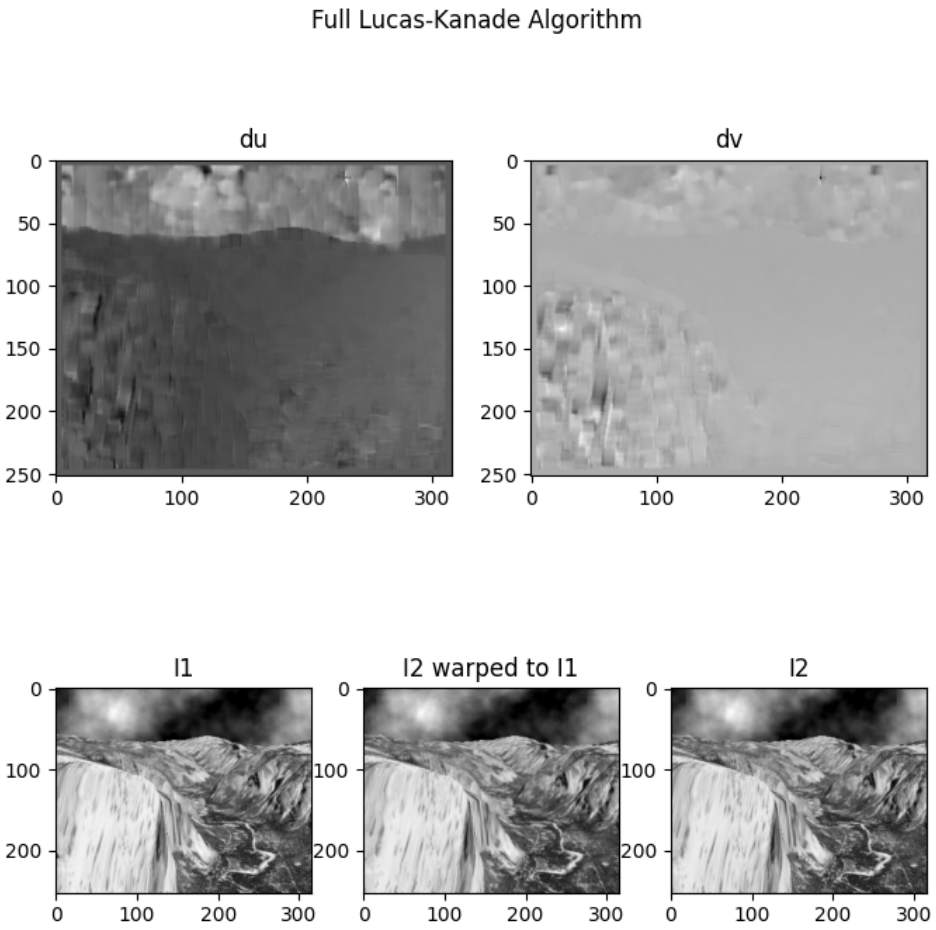

Figure 2: Full LK algorithm result.

Parameters chosen: `WINDOW_SIZE_RIVER = 10`, `MAX_ITER_RIVER = 10`, `NUM_LEVELS_RIVER = 5`.

Results (interest-region MSE):

- Original frames: 46.61
- After one LK step: 300.66 (ratio 0.16)
- After full pyramidal LK: 45.49 (ratio 1.02)

Explanation. The full LK runs the step over an image pyramid: coarse (small) levels capture the large motion that a single full-resolution step cannot, and each finer level refines the flow with

sub-pixel corrections. So the warped I_2 now matches I_1 even in the fast-moving sky, and the MSE drops from 300 (one step) to 45.5, slightly below the original 46.6 (ratio > 1). The remaining error is mostly the sky itself, which physically changes between frames and cannot be matched by any warp.

This is why `3_after_full_lk.gif` looks better: the two frames are now aligned, so the flicker/jitter between them almost disappears, unlike the one-step gif where the sky region was broken.

Section 3 – Video Stabilization

Q8

Run `main_tau_video.py` to create `ID1_ID2_stabilized_video.avi`. Explain the result you obtained. What happened to the MSE of the video frames?

Answer.
Parameters: `WINDOW_SIZE_TAU = 5`, `MAX_ITER_TAU = 3`, `NUM_LEVELS_TAU = 3`.

The video is stabilized: the camera shake is removed and the scene stays almost locked to the position of the first frame. The mean MSE between consecutive frames drops a lot, from 60.23 to 25.41. The borders, however, look bad: warping shifts each frame, and strange effect is appearing in the borders.

Video	Mean MSE between frames
Original video	60.23
LK stabilized	25.41

Table 1: Reference: 60.23 → 26.73 (reduction ≥ 40%).

Q11

Implement `lucas_kanade_faster_video_stabilization`. The runtime should decrease by at least 30%. The MSE between frames should be lower than the original video MSE by at least 30%.

Results.

Video	Mean MSE	Runtime (s)
Original	60.23	—
LK stabilized	25.41	320.91
Faster LK stabilized	37.52	191.03

Table 2: Reference MSE: 60.23 / 26.73 / 29.03. Runtime reduction ≥ 30%.

Q12

Implement `lucas_kanade_faster_video_stabilization_fix_effects`. This function fixes border effects of the Lucas Kanade implementation by cutting out a constant portion of the image.

Results.

Video	Mean MSE
Original	60.23
Faster LK + borders cut	31.91

Table 3: Reference: 60.23 → 24.39.